# Enabling Transparent Integrity Auditing and Secure Deduplication Over Encrypted Cloud Storage Based on Blockchain

Yang Yang, Tianqing Zhu, Wenting Shen, Yanjiao Chen, Shanshan Li, Fei Chen, and Jing Chen

***Abstract*—Cloud storage brings significant advantages in the era of data explosion. Secure deduplication significantly promotes cloud storage efficiency while preserving user privacy. At the same time, cloud storage also faces the risk of data loss, so auditing its integrity becomes imperative. Under this circumstance, researchers have begun to integrate secure deduplication with integrity auditing. Unfortunately, the existing works are still unsatisfactory in terms of protecting ownership privacy and resisting deduplication pattern degradation. To address these problems, this paper proposes a blockchain-based solution that integrates secure deduplication and integrity auditing over encrypted cloud storage. Specifically, we design a random masking technique to generate unlinkable integrity proofs for different users, thereby protecting ownership privacy during auditing. We further construct a blockchain-based transparent deduplication mechanism to record and maintain the ownership relationship between ciphertext files and their owners, preventing deduplication pattern degradation when users go offline. In addition, we propose a lightweight hash-based probabilistic proof of ownership to resist the duplicate-faking attack while avoiding full-file ownership verification. Finally, rigorous theoretical analysis is provided to demonstrate the security guarantees of our proposed solution, and massive experimental results justify its performance superiority.**

***Index Terms*—Secure deduplication, integrity auditing, cloud storage, blockchain.**

Received 7 February 2026; revised 8 June 2026; accepted 15 July 2026. Date of publication 17 July 2026; date of current version 1 September 2026. This work was supported in part by the Shandong Provincial Natural Science Foundation under Grant ZR2025MS1006, in part by the Youth Innovation Technology Project of Higher School in Shandong Province under Grant 2023KJ365, in part by the Zhejiang Key Laboratory of Electrical Technology and System on Renewable Energy, the China Postdoctoral Science Foundation (Special Grant) under Grant 2025T180418, in part by the Postdoctoral Fellowship Program (Grade C) of China Postdoctoral Science Foundation under Grant GZC20233144, in part by the Postdoctor Project of Hubei Province under Grant 2024HBBHCXB092, in part by the Fundamental Research Funds for the Central Universities, Zhongnan University of Economics and Law under Grant 2722025BJ064, and in part by the National Natural Science Foundations of China under Grant 62302406 and Grant 62272318. *(Corresponding author: Wenting Shen.)*

Yang Yang and Shanshan Li are with the School of Information Engineering, Zhongnan University of Economics and Law, Wuhan 430073, China (e-mail: yaoyuandepiaoxue@126.com; ershineli@126.com).

Tianqing Zhu is with the Faculty of Data Science, City University of Macau, Macau, China (e-mail: tqzhu@cityu.edu.mo).

Wenting Shen is with the College of Computer Science and Technology, Qingdao University, Qingdao 266071, China (e-mail: shenwentingmath@163.com).

Yanjiao Chen is with the College of Electrical Engineering, Zhejiang University, Hangzhou 310027, China (e-mail: chenyanjiao@zju.edu.cn).

Fei Chen is with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China (e-mail: fchen@szu.edu.cn).

Jing Chen is with the Computer School, Wuhan University, Wuhan 430072, China (e-mail: chenjing@whu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3714758

## I. Introduction

In THE era of Big Data, cloud storage has become increasingly critical. It allows users to benefit significantly from outsourcing large-scale data to cloud servers [1]. However, it also inevitably introduces severe concerns about data security since users lose direct control over their outsourced data [2]. On the one hand, once the cloud server provider (CSP) is compromised, the content of user data will be exposed, thereby infringing on user privacy. To address this problem, the most straightforward approach is to require users to encrypt their data before uploading it, which is widely adopted in practice. On the other hand, once the CSP loses user data due to hardware failures and software bugs, it might try its best to cover the data abnormality for its own interest, thereby raising user concern on the integrity of outsourced data. To resolve this problem, plenty of integrity auditing schemes are presented, in which users have to periodically perform integrity auditing tasks by themselves or through a third-party auditor (TPA) [3], [4], [5], [6], [7], [9], [10], [11], [12], [13], [14], [15], [16]. Another focal point is to reduce unnecessary cloud storage costs as much as possible because different users may upload the same data to a cloud server, leading to redundant storage expenses [17], [18]. Under this circumstance, CSPs are required to perform the deduplication on user outsourced data [19], [20], [21], [22], [23], [24], [25], [26]. Taking into account the above urgent requirements of privacy protection, integrity auditing and data deduplication, it is essential to simultaneously support both integrity auditing and secure deduplication over encrypted cloud storage.

Up to now, some efforts have combined existing integrity auditing and secure deduplication techniques [27], [28], [29], [29], [30], [31], [32]. These studies mainly concentrate on traditional security issues, such as data privacy, data unforgeability, and achieve resistance to the duplicate-faking attack (DFA) through hash-based proof of ownership (PoW). However, arising from integrating integrity auditing and secure deduplication, there are several emerging security issues, which are often not considered. One is the protection of the ownership privacy (OP) of the users [33]. The leakage of the OP indicates that a TPA and other users are able to learn the following knowledge: i) which files belong to a user; ii) which users have the same file. As more and more users choose TPA to initiate integrity auditing to relieve the local workloads, the protection of the OP becomes increasingly urgent. To meet this challenge, a recent integrity

auditing protocol [33] is designed based on the re-signature technique. Unfortunately, the TPA is able to crack the ephemeral re-signature key used to generate the integrity proof by the CSP through the attack in [34], leading to the ciphertext of user data blocks disclosed. In such a way, the TPA can easily determine which files belong to a user. Also, the TPA can determine which users own an identical file through auditing the same blocks during the different auditing delegation tasks. This means that the protocol of [33] does not protect the OP of the users as it claims. As a result, it is challenging to design an integrity auditing mechanism that can truly protect the OP of the users.

The other is the resistance to the deduplication pattern degradation (DPD). Considering the deduplication pattern is composed of the identifiers of users uploading the same file, the DPD means that the CSP aims to deceive its users by providing false duplication-checking results, degrading the deduplication pattern. For example, in multi-tenant cloud storage environments, popular datasets, virtual machine images, or software packages are often uploaded by multiple users. Although data deduplication enables the CSP to store only a single copy and reduce storage costs, a profit-driven CSP may intentionally report that no duplication is detected and require each user to upload the file separately without offering the appropriate discounts. As a result, users are charged full storage fees, while the CSP benefits from reduced actual storage consumption. This behavior undermines the economic fairness and transparency promised by deduplication-based cloud services, highlighting the practical need to defend against DPD in real-world deployments. What is more, as the number of users uploading the same file increases, the CSP has more incentives to initiate the DPD, making resistance to the DPD even more pressing. Although a recent scheme [35] was proposed to resist the DPD, which can prevent users from being deceived by the CSP during data duplication checks, it relies on two premises: one is that users must always be online; otherwise, there is a possibility of missed DPD detections; the other is that the CSP and its users would not dispute the verification results of the DPD; otherwise, it cannot effectively arbitrate the dispute. These two premises can hardly be guaranteed since the behavior of both the CSP and its users is unpredictable. To address this problem, it is meaningful to design a blockchain-based mechanism to achieve cheating resistance to the DPD, in which the data duplication-checking results can be credibly accepted by both the CSP and its users.

Based on the above discussion, although the existing schemes can effectively address traditional security concerns, none of them can adequately meet emerging security challenges: the protection of the OP and the resistance to the DPD. In this context, we first propose a novel integrity auditing mechanism in which a random masking technique is used to blind the file tag and the integrity proof. To be specific, for a data file, each user uses different file tags for delegating audit tasks to the TPA, and the CSP feeds back distinct blinding integrity proofs to the TPA even in front of the same auditing request. More importantly, since our integrity auditing mechanism does not expose the random masking value as a multiplier to the TPA, it can prevent the OP of the users from the attack in [34] based on the greatest common divisor. We then design a blockchain-based transparent deduplication mechanism, where the ownership relationship between the ciphertext-file hash and the set of blinding user identities of the corresponding file owners is published on the blockchain. To be specific, our transparent deduplication mechanism requires the cloud and its users to jointly maintain the deduplication pattern on the blockchain, which thus can fundamentally solve the problem of mutual trust between the CSP and its users regarding the deduplication pattern. In addition, since the on-chain deduplication pattern is immutable, users can go offline after confirming it. Finally, we integrate the above two mechanisms into one protocol of integrity auditing and secure deduplication over encrypted cloud storage.

Our main contributions are listed as follows.

- We propose a blockchain-based protocol of transparent integrity auditing and secure deduplication over encrypted cloud storage (BTAD), which is the first time to truly realize the protection of the OP and the resistance to the DPD. Furthermore, we also propose a hash-based probabilistic PoW in order to resist the DFA.
- We propose an integrity auditing mechanism by performing the random masking technique on the file tags and the integrity proofs in order to prevent the OP of the users from the TPA, and a transparent deduplication mechanism by using the on-chain deduplication pattern jointly maintained by both the CSP and its users so as to resist the DPD of the CSP.
- We carry out a comprehensive analysis of the BTAD from both the theoretical and experimental perspectives. In comparison to the recent state-of-the-art, our BTAD achieves enhanced security while also exhibiting good performance in terms of storage, communication and computation costs.

The rest of this paper is organized as follows: Section II describes the related work. Section III shows the preliminaries of this paper. Section IV illustrates the proposed BTAD. Section V gives the theoretical analysis. Section VI provides the performance evaluation. At last, Section VII presents the conclusion.

## II. Related Work

### A. Independently Evolved Solutions

Before studying the integration of integrity auditing and secure deduplication, these two types of techniques have respectively received extensive research. On the one hand, for integrity auditing, Ateniese et al. proposed a groundbreaking solution based on provable data possession [3], enabling users to find the abnormality of cloud storage without downloading the entire data. Following Ateniese et al.'s breakthrough, plenty of integrity auditing schemes were proposed to enhance its functionality, such as dynamic updates of cloud storage [4], [5], [6], resistance to key-exposure [7], [8], blockchain-based reliability of auditing results [9], [10], compression of cloud storage [11], [12], user revocation [13], [14], auditing incentives [15], [16], etc. On the other hand, for secure deduplication, Douceur et al. first proposed a concept called convergent encryption (CE) [19], which takes the hash value of the data as its encryption key so that different users can encrypt an identical file into the same ciphertext without key exchanges. Inspired by CE, Bellare et al. formalized a new primitive message-locked encryption

(MLE) [20], which is widely adopted in the design of secure deduplication solutions [21], [22], [23], [24], [25], [26].

### B. Integrated Solutions

The seminal integrated solution of simultaneously supporting integrity auditing and secure deduplication was proposed in [27]. Its improvement in storage efficiency mainly stems from deduplicating both plaintext blocks and their tags on the CSP. However, it has two disadvantages: one is that all of the users have to calculate the block tags of the same file, sacrificing the local computation efficiency; the other is that users directly upload the plaintext to the CSP, making their data privacy disclosed. Faced with the above two challenges, researchers started to shift their focus towards studying the integration of integrity auditing and secure deduplication over encrypted cloud storage. Initially, Li et al. proposed a solution that simultaneously supports integrity auditing and encrypted deduplication, in which the tags of ciphertext blocks are generated by a completely trusted proxy server [28]. However, implementing such a completely trusted proxy server is practically quite expensive due to insecure public networks. To address this problem, Liu et al. proposed a solution without the help of a completely trusted proxy server [29]. This scheme can directly eliminate the duplicate encrypted copies by using the MLE. Unfortunately, it requires that the TPA is fully trusted during integrity auditing, which is hard to guarantee due to the unpredictable behavior of the TPA.

### C. Blockchain-Based Integrated Solutions

In order to avoid introducing any fully trusted component, which might lead to single point of failure, an increasing number of researchers begun to explore blockchain-based approaches to simultaneously enable integrity auditing and encrypted deduplication. Yuan et al. proposed a blockchain-based solution [30], in which the blockchain is used to deploy smart contracts [36] for performing integrity auditing and fair arbitration. Unfortunately, its data deduplication strategy is cloud-side, which requires all of the users to respectively calculate and upload the encrypted blocks and their tags for the same file, resulting in a significant amount of redundant communication and computation. In what follows, Halevi et al. initially proposed a novel concept called PoWs based on Merkle hash trees [37], which realizes user-side data deduplication. Then, many blockchain-based solutions were proposed based on the PoW, in which only the initial user needs to upload the encrypted file blocks and their tags, and subsequent users do not need to upload the same data again. Specifically, Tian et al. proposed a solution that can deal with a single point of failure (SPOF) by using two CSPs [31], and its blockchain is used to record file index tables and integrity auditing logs. Inspired by the double-copy storage model in [31], Zhang et al. constructed a solution that can protect user data from SPOF in the scenario of off-chain distributed storage [32], and its blockchain is used to deploy smart contracts for performing the PoW and the auditing. However, the leakage of the OP and the attack from the DPD are not considered in [31], [32]. To protect the OP of the users, Song et al. investigated a solution by using the re-signature technique [33], and its blockchain is used to record integrity auditing logs. Unfortunately, Song et al.'s scheme cannot protect the OP of the users against the attack in [34]. To resist the DPD of the CSP, Li et al. designed a solution based on the transparent deduplication pattern [35], and its blockchain is used to record the proofs of the deduplication pattern and integrity auditing. Nevertheless, Li et al.'s scheme has two limitations: one is that users must be online to prevent the DPD; the other is that the CSP and its users are not allowed to challenge the verification results of the DPD, making it less convincing.

As summarized in Table I, although the existing schemes of simultaneously supporting integrity auditing and secure deduplication can effectively address the traditional security issues: data privacy, data unforgeability, and resistance to DFA, they cannot perfectly handle emerging security issues: protection of the OP and resistance to the DPD.

## III. Preliminaries

This section first illustrates the system model of our proposed BTAD and then introduces its threat model, design goals and technical background.

### A. System Model

A protocol of blockchain-based transparent integrity auditing and secure deduplication over encrypted cloud storage contains four entities: users, CSP, TPA, and the blockchain, which are illustrated as follows:

- *Users:* are to upload the ciphertext files on the CSP. Also, they entrust a TPA to verify the integrity of their outsourced data. Due to the possibility that different users might outsource the same file, there are two types of users: one is the initial uploader of outsourcing a new file to the CSP; the other is the subsequent uploader of outsourcing an existing file.
- *CSP:* is to provide storage services for its users, which also handles data deduplication and generates integrity proofs during the auditing process.
- *TPA:* is to receive auditing delegations from users and perform cloud storage auditing tasks. For each audit, it issues proof of data integrity on the blockchain for public supervision, including challenge sequence, cloud proof, and verification result.
- *Blockchain:* is to record the deduplication pattern and the proofs of data integrity.

*Definition III.1:* A protocol of BTAD is composed of five algorithms as follows:

- *System Setup:* This algorithm produces the parameters required for the subsequent algorithms.
- *Key Generation:* This algorithm enables users to generate MLE keys, including block encryption keys and block signing key.
- *Data Outsource:* This algorithm allows users to outsource the ciphertext of the targeted file to the CSP in a deduplicable way and enables the CSP to publish the deduplication pattern of the targeted file on the blockchain.
- *Integrity Auditing:* This algorithm lets the TPA perform integrity auditing tasks by interacting with the CSP and publishing data integrity proofs on the blockchain.

TABLE I
THE COMPARISON OF FUNCTIONALITY AND SECURITY AMONG DIFFERENT BLOCKCHAIN-BASED INTEGRATED SOLUTIONS

| Protocol | Data privacy | User offline | Resisting DFA | Resisting DPD | Protecting OP | File-level deduplication | Block-level deduplication |
|---|---|---|---|---|---|---|---|
| Yuan et al.[30] | Yes | Yes | No | No | No | Yes | Yes |
| Tian et al.[31] | Yes | Yes | Yes | No | No | Yes | Yes |
| Zhang et al.[32] | Yes | Yes | Yes | No | No | Yes | Yes |
| Song et al.[33] | Yes | Yes | Yes | No | No | Yes | Yes |
| Li et al.[35] | Yes | No | Yes | Yes | No | Yes | No |
| The proposed BTAD | Yes | Yes | Yes | Yes | Yes | Yes | Yes |

- *Data Retrieval:* This algorithm allows users to recover the plaintext data from the CSP.

### B. Threat Model

On the basis of the system model in the above subsection, we provide the corresponding threat model. In practical cloud storage environments, CSPs and TPAs are usually constrained by commercial contracts, legal liabilities, and reputation concerns. Therefore, they may not always have strong incentives to actively misbehave. Nevertheless, for high-value and privacy-sensitive outsourced data, relying only on contractual trust may be insufficient, especially in multi-tenant cloud storage environments where deduplication patterns are not transparent to users. A profit-driven CSP may still have incentives to hide the actual deduplication relationship and charge users without offering the expected deduplication benefit, while a curious or compromised TPA may infer ownership relationships from auditing logs. Therefore, our threat model should be understood as a conservative security-oriented model that captures possible risks caused by economic incentives, insider misbehavior, compromise, or lack of transparency.

- *Semi-honest users:* Users may pry into the OP of other users from the on-chain auditing logs, which contain the information of file owners. Users may also launch DFA, making their subsequent users lose data.
- *Semi-honest CSP:* To protect its reputation, the CSP may attempt to deceive the TPA by using counterfeit data when its storage is abnormal. Furthermore, the CSP may cheat users by providing false duplication-checking results to increase its revenue. However, the CSP is assumed to behave honestly during the system setup and initial verification phases. This assumption is reasonable because these phases are necessary for the normal operation of the whole system. Specifically, if the CSP generates invalid public parameters during system setup, the subsequent protocols cannot be correctly executed, making the service unusable and only harming the CSP's reputation and revenue. Similarly, if the CSP deviates from the prescribed initial verification procedure, legitimate upload requests may fail, forcing data owners to repeatedly upload data and increasing system overhead without bringing any tangible benefit to the CSP. Therefore, a rational CSP has little incentive to deviate from the protocol in these phases, while it may still behave semi-honestly in later storage, deduplication, and auditing processes.
- *Semi-honest TPA:* For the sake of interest, the TPA may provide biased auditing results that cause user data exceptions not to be disclosed in time. The TPA may also attempt to infer the OP of users from its auditing logs and the on-chain deduplication pattern.

Besides semi-honest users, CSP and TPA may also be curious about the plaintext contents of cloud storage and attempt to crack their ciphertexts.

It should be noted that this work considers a semi-honest and non-colluding adversarial model. The proposed random masking mechanism aims to prevent the TPA, other users, and public observers from inferring file ownership relationships from auditing proofs and on-chain records. It does not aim to hide file ownership from the CSP, since the CSP, as the storage service provider, must maintain certain ownership information to support deduplication, ownership verification, auditing, and data retrieval. In particular, the blinding file tag is transmitted to the CSP during integrity auditing so that the CSP can locate the audited file and generate the corresponding proof. Therefore, collusion between the CSP and the TPA is beyond the threat model considered in this paper. If the CSP shares its privileged storage-side information or secret parameter with the TPA, the TPA may obtain additional ownership information with the help of the CSP. Defending against such collusion would require stronger mechanisms, such as threshold-managed secret parameters, multiple non-colluding CSPs, or trusted execution environments, which are left as future work.

### C. Design Goals

In view of the above security threat, the design goals of our BTAD are as follows:

- *Functionality:* The scheme has two fundamental functionalities: integrity auditing and secure deduplication. To be specific, the scheme simultaneously allows users to perform integrity auditing on their cloud storage and the CSP to perform deduplication on both ciphertext blocks and their tags.

- *Security:* The scheme has five security goals: data privacy, unforgeability, OP protection, and resistance to DFA and DPD. 1) Data privacy means that the plaintexts of a user are required to be hidden from other users, the CSP, and the TPA; 2) Unforgeability indicates that the CSP cannot cheat the TPA with the forged user data when the real user data is lost; 3) OP protection ensures that entities other than the CSP cannot ascertain which files belong to a user and which users possess the same files; 4) Resistance to DFA means that the subsequent users are prevented from losing their data in front of DFA; 5) Resistant to DPD is that the subsequent users are protected from paying storage fees without the deserved discount.
- *Efficiency:* The scheme should have the lowest possible storage, communication, and computation costs.

### D. Technical Background

*1) Hash and Convergent Encryption:* As a special case of MLE, HCE is also a symmetric encryption algorithm whose key is the hash value of the input data [20]. In this way, different users with identical data can obtain the same ciphertext, so HCE is widely adopted in ciphertext deduplication. Formally, HCE can be defined as follows:

- $\mathsf{KeyGen}(\lambda, m_i) \rightarrow k_i$*:* is to output a convergent key $k_i$ by inputting the security parameter $\lambda$ and the plaintext $m_i$.
- $\mathsf{Encrypt}(k_i, m_i) \rightarrow c_i$*:* is to output the ciphertext $c_i$ by encrypting the plaintext $m_i$ with the key $k_i$.
- $\mathsf{Decrypt}(k_i, c_i) \rightarrow m_i$*:* is to output the plaintext $m_i$ by decrypting the ciphertext $c_i$ with the key $k_i$.

*2) Blockchain:* A blockchain is a distributed ledger composed of a sequence of blocks that are collectively generated and maintained by multiple decentralized nodes under a common consensus mechanism [38]. Specifically, each block contains a set of essential metadata, including a timestamp, a nonce, and a collection of transactions, and is cryptographically linked to its preceding block via a hash pointer. This chained structure ensures that any modification to a historical block would invalidate all subsequent blocks, thereby endowing the blockchain with fundamental properties such as immutability, traceability, and tamper resistance.

As a prominent application of blockchain technology, Ethereum extends the basic ledger functionality by enabling the execution of programmable logic through smart contracts. In Ethereum, a smart contract refers to a piece of program code deployed on the blockchain that is collectively agreed upon and authenticated by its participants. Once deployed, the contract is automatically executed by the blockchain network in response to predefined transactions, without reliance on a trusted third party.

*3) Bilinear Pairing:* Given two multiplicative cyclic groups $G_1$ and $G_2$ with the prime order $p$ and a bilinear mapping $e : G_1 \times G_1 \rightarrow G_2$, a bilinear pairing is a special mapping that connects two elements from the source group $G_1$ to an element in the target group $G_2$, while preserving algebraic relationships in a structured manner [39]. Such mappings play a fundamental role in pairing-based cryptographic constructions.

Formally, the bilinear pairing $e$ satisfies the following properties:

- *Bilinearity:* $e(x^\alpha, y^\beta) = e(x, y)^{\alpha\beta}$ for arbitrary $x, y \in G_1$ and $\alpha, \beta \in Z_p$.
  This property indicates that the pairing preserves exponentiation across groups, enabling multiplicative relationships in $G_1$ to be translated into exponentiation in $G_2$. Bilinearity is the core feature that allows pairings to support advanced cryptographic functionalities such as aggregation, delegation, and verification.
- *Non-degeneracy:* $e(g, g) \neq 1$ for a generator $g \in G_1$.
  This property guarantees that the pairing does not collapse all input pairs into the identity element of $G_2$, ensuring that the mapping is non-trivial and information-preserving.
- *Computability:* For arbitrary $x, y \in G_1$, $e(x, y) \in G_2$ can be computed efficiently.
  This property ensures the practical applicability of bilinear pairings in real-world cryptographic systems.

In addition, the security of pairing-based schemes typically relies on the hardness of certain number-theoretic problems. One such fundamental problem is the Discrete Logarithm Problem (DLP), which is defined as follows: given $g, g^x \in G$, the DLP is to compute $x \in Z_p$ [40]. The intractability of the DLP in cyclic groups underpins the security guarantees of many cryptographic protocols built upon bilinear pairings.

The main utilization of bilinear pairing in this paper is briefly described as follows.

- The design of block tag enables the integrity of encrypted blocks to be verified.
- The design of the blinding user identity enables the verification of file ownership, indicating which user one file belongs to.
- The design of the auditing request enables the audited file to be uniquely identified and the integrity of the file blocks to be verified.
- The basis of the security proof for unforgeability.

## IV. The Proposed Protocol

### A. Overview

The workflow of our scheme is depicted in Fig. 1. Before file uploading, users are required to compute file block encryption keys by performing key generation. When uploading a file, a user is required to detect file duplication by examining whether the hash value of its ciphertext exists in on-chain deduplication pattern. If not, the user is considered as an initial uploader and outsources the encrypted block set $C$, the block tag set $T$, the ciphertext $Ck$ of block encryption keys, the hash value $h_e$ of the ciphertext file, the file tag $g^k$ and the blinding user identity $h^k$. The CSP publishes $h_e$ and $h^k$ to the smart contract for updating the deduplication pattern. Otherwise, the user becomes a subsequent uploader who uploads $h_e$ and $h^k$ only in the case that his/her PoW passes the verification of the CSP. Through smart contract, the CSP appends the blinding user identity $h^k$ of the subsequent uploader on the blockchain according to the hash value $h_e$ of the duplicated ciphertext file. One thing to note is that users can check whether the CSP honestly publishes their blinding identities on the blockchain at any time. For data reconstruction, a user first downloads the encrypted file

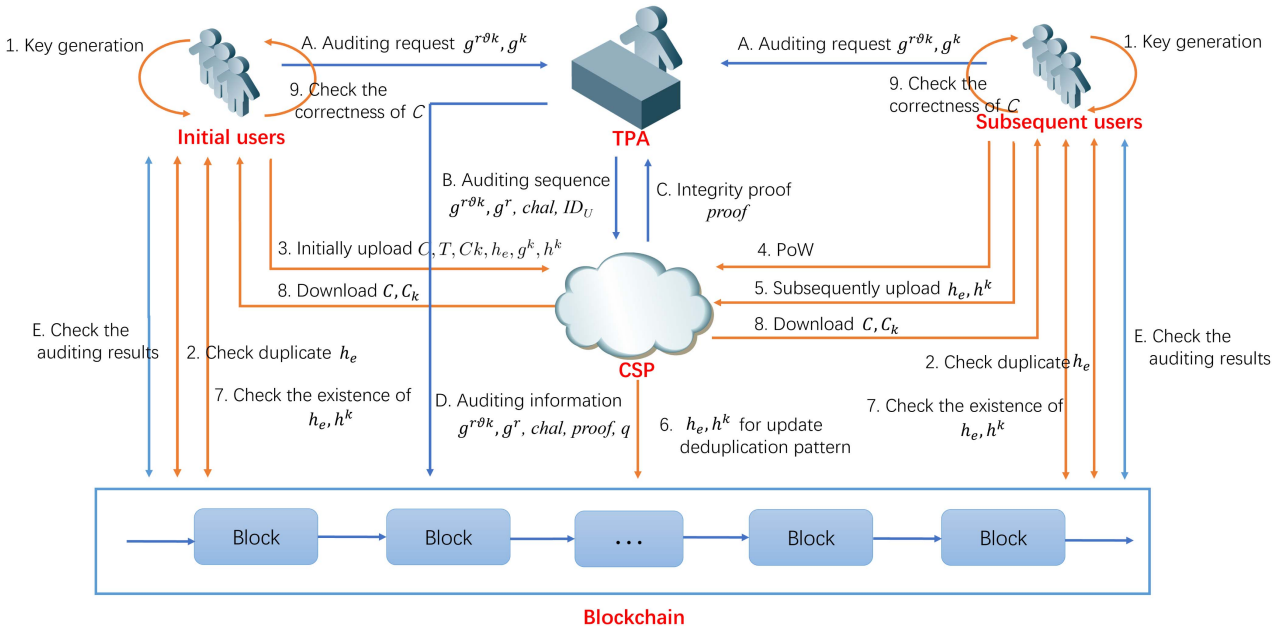


Fig. 1. The workflow of our scheme.

block set $C$ and the ciphertext $Ck$ of block encryption keys and then deciphers the ciphertext blocks using the decrypted block encryption keys. Another thing to note is that users can verify the correctness of decrypted plaintext files by comparing their hash values with their locally maintained ones.

During the integrity auditing, a user transmits an auditing request, including a blinding file tag $g^{r\vartheta k}$ and the related blinding parameter $g^r$, to the TPA to delegate the integrity auditing task. The TPA transmits an auditing sequence to the CSP, which is composed of the blinding file tag $g^{r\vartheta k}$, the blinding parameter $g^r$, the challenge $chal$, and the identity $Ind_U$ of the audit-delegating user. Once receiving the auditing sequence, the CSP computes the integrity proof $proof$ based on the audited user's stored data, which is fed back to the TPA. Subsequently, the TPA outputs the verification result $q$, and publishes the total auditing information ($g^{r\vartheta k}$, $g^r$, $chal$, $proof$, $q$) on the blockchain. Finally, the user can check the correctness of the on-chain auditing results from time to time.

It is worth noting that the proposed design introduces additional cryptographic and system components because our goal is not merely to hide the communication between users and the TPA. Encrypted or anonymous communication channels can protect the transmission process, but they cannot prevent ownership leakage from deterministic auditing proofs, on-chain auditing logs, or deduplication patterns. Moreover, such channels cannot prevent a profit-driven CSP from hiding the real deduplication relationship, nor can they provide immutable evidence for users once they go offline. In contrast, the bilinear-pairing-based auditing mechanism enables public integrity verification over encrypted data, the random masking technique makes integrity proofs unlinkable across different users, and the blockchain-based transparent deduplication mechanism records and maintains the ownership relationship between ciphertext-file hashes and blinding user identities in a tamper-resistant manner. Therefore, the introduced complexity is necessary to simultaneously achieve secure deduplication, integrity auditing, ownership privacy protection, resistance to duplicate-faking attacks, resistance to deduplication pattern degradation, and transparent accountability.

### B. Protocol Construction

This subsection focuses on the detailed description of the proposed BTAD from the perspectives of SystemSetup, KeyGeneration, DataOutsourcing, Integrityauditing and Dataretrieval, and the notations involved are summarized in Table II.

1) *System Setup:* Given a security level λ, the CSP initializes the whole system as follows.

- Generate two multiplicative cyclic groups $G_1$ and $G_2$ with the prime order $p$ and a bilinear mapping $e : G_1 \times G_1 \rightarrow G_2$.
- Choose two random hash functions $H_1 : \{0,1\}^* \rightarrow G_1$ and $H_2 : \{0,1\}^* \rightarrow Z_p$.
- Choose a generator $g$ and random elements $u_1, u_2, \ldots, u_s$ from $G_1$.
- Choose two pseudo-random functions $\pi_1 : \{1,2,\ldots,n\} \times Z_p \rightarrow \{1,2,\ldots,n\}$ and $\pi_2 : \{1,2,\ldots,n\} \times Z_p \rightarrow Z_p$.
- Choose a random element $\vartheta$ from $Z_p$.

At last, the CSP publishes $\{u_1, u_2, \ldots, u_s, \pi_1, \pi_2, e, g, g^\vartheta, H_1, H_2\}$ as the public parameters, and holds $\vartheta$ as the secret parameter.

2) *Key Generation:* Given a data file $F = m_1||m_2||\cdots||m_n$, the user produces the corresponding encryption keys.

- The user computes the file hash value as $k = H_2(F)$, which is taken as the block signing key.
- To relieve the local storage burden, the user only keeps the block signing key $k$ and the file identifier at local.

3) *Data Outsourcing:* When the data file $F$ is to be outsourced, the user first uses the block signing key $k$ to encrypt $F$ into $E$ by computing $E = Enc(k; F)$, where $Enc$ is a symmetric encryption algorithm. Then, the user computes $h_e = H_2(E)$, which is used as the identifier of the encrypted file. Next, the

TABLE II
PROTOCOL NOTATIONS

| Symbol | Description |
|---|---|
| $F$ | Original data file outsourced by the user |
| $m_i$ | The $i$-th plaintext data block of the file $F$ |
| $C$ | Encrypted file consisting of the ciphertext blocks $\{c_i\}$ |
| $c_i$ | Ciphertext of the $i$-th data block |
| $c_{i,j}$ | The $j$-th segment of the ciphertext block $c_i$ |
| $n$ | Number of data blocks in the file $F$ |
| $s$ | Number of segments in each ciphertext block |
| $k$ | Block signing key derived from the file hash |
| $k_i$ | Encryption key of the plaintext block $m_i$ |
| $Ck$ | Encrypted block encryption keys $\{k_i\}_{i=1}^n$ |
| $ID_U$ | Identity of the data owner (user) |
| $h_e$ | Identifier of the encrypted file $E$ |
| $h^k$ | Blinding user identity bound to identity $ID_U$ |
| $\sigma_i$ | Authentication tag of the ciphertext block $c_i$ |
| $D$ | On-chain mapping table storing file ownership |
| $r$ | Random blinding parameter selected by the user |
| $g$ | Generator of the cyclic group $G_1$ |
| $g^r$ | Blinding parameter transmitted to the TPA |
| $g^{r\vartheta k}$ | Blinding file tag used in integrity auditing |
| $chal$ | Integrity auditing challenge generated by the TPA |
| $c$ | Number of challenged blocks in an audit |
| $\tau$ | Random seed used to generate challenge indices |
| $\pi_1(\cdot)$ | Pseudo-random permutation |
| $\pi_2(\cdot)$ | Pseudo-random function |
| $a_i$ | Index of the $i$-th challenged ciphertext block |
| $b_i$ | Challenge coefficient associated with block index $a_i$ |
| $x$ | Random masking value selected by the CSP |
| $\mu_j$ | Linear combination of ciphertext segments in the proof |
| $v$ | Aggregated authentication tag in the integrity proof |
| $\zeta$ | Aggregated hash-based component in the integrity proof |
| $proof$ | Integrity proof generated by the CSP |
| $q$ | Integrity auditing result ($q = 1$ for success, 0 otherwise) |

**Algorithm 1:** Data Outsourcing.

**Input:** User file $F = m_1\|m_2\|\cdots\|m_n$, user identity $ID_U$, user block signing key $k \leftarrow H_2(F)$, system parameters $\{u_1, u_2, \ldots, u_s, \pi_1, \pi_2, e, g, g^{\vartheta}, H_1, H_2\}$.

1: User encrypts $F$ to obtain $E \leftarrow Enc(k, F)$.
2: User computes the file identifier $h_e \leftarrow H_2(E)$.
3: **if** $h_e \notin D$ **then**
4: **for** $i = 1$ to $n$ **do**
5: User derives the block encryption key $k_i \leftarrow H_2(m_i)$.
6: User encrypts block $c_i \leftarrow Enc(k_i, m_i)$.
7: **end for**
8: User encrypts the block keys $Ck \leftarrow Enc(k, k_1, \ldots, k_n)$.
9: **for** $i = 1$ to $n$ **do**
10: User generates the tag $\sigma_i \leftarrow (H_1(h_e\|i) \cdot \prod_{j=1}^{s} u_j^{c_{i,j}})^k$.
11: **end for**
12: User computes the blinding user identity $h^k \leftarrow H_1(ID_U)^k$.
13: User uploads $\{c_i\}_{i=1}^n$, $\{\sigma_i\}_{i=1}^n$, $Ck||h_e||g^k||h^k$ to CSP.
14: **if** $e(h^k, g) = e(h', g^k)$ **then**
15: CSP submits $(h_e, h^k)$ to *SC*
16: *SC* registers a mapping $(h_e \mapsto h^k)$ into $D$ after ensuring the submission from its deployer (i.e., CSP).
17: User checks the consistency of the retrieved $h^k$ from $D$ using $h_e$ as the lookup key.
18: **else**
19: CSP reports the failure.
20: **end if**
21: **else**
22: User requests a challenge $chal = \{c, \tau\}$ from CSP.
23: User responds with $g^k||h_\tau||h^k$ as the PoW to CSP, where $h_\tau \leftarrow H_2(\tau||\{c_{\pi_1(i,\tau)}\}_{1\le i\le c})$ and $h^k \leftarrow H_1(ID_U)^k$.
24: **if** PoW verification succeeds **then**
25: Repeat Steps 15–17, in which $h_e$ is retrieved by CSP using $h_k$ as the index.
26: **else**
27: CSP refuses to accept user as the owner of $F$.
28: **end if**
29: **end if**

user checks if $h_e$ exists in the mapping table $D$ to which users a file belongs on the blockchain, where the key in $D$ stores the identifier of a file and its value stores the identifiers of all these file owners. If $h_e$ does not exist, the user will execute the initial upload operation. Otherwise, the user will execute the subsequent upload operation. To facilitate understanding, we summarize the pseudo-algorithm of dataoutsourcing in Algorithm 1 and its detailed procedure is described as follows.

*Initial upload:* For the file $F$, the user performs the initial upload as follows.

- The initial uploader first computes each block encryption key as $k_i = H_2(m_i)$, where $1 \le i \le n$. Then, the initial user utilizes the key $k_i$ to encrypt each block $m_i$ into $c_i$ by computing $c_i = Enc(k_i; m_i)$, where $1 \le i \le n$.
- The initial user partitions each ciphertext block $c_i$ into $s$ segments, i.e., $c_i = \{c_{i,1}, c_{i,2}, \ldots, c_{i,s}\}$, and computes its tag as $\sigma_i = (H_1(h_e||i)\prod_{j=1}^{s} u_j^{c_{i,j}})^k$ by using the block signing key, where $1 \le i \le n$.
- The initial user utilizes the key $k$ to encrypt the block encryption keys $\{k_1, k_2, \ldots, k_n\}$ into $Ck$ through computing $Ck = Enc(k; k_1, k_2, \ldots, k_n)$.
- The initial user uploads the encrypted file $C = \{c_i\}_{1\le i\le n}$ and its tag set $T = \{\sigma_i\}_{1\le i\le n}$ to the CSP. The initial user also uploads the file supplementary information $Ck||h_e||g^k||h^k$ to the CSP, where $h^k = H_1(ID_U)^k$ is used

to indicate that $F$ belongs to the user with the identity $Ind_U$.

- After receiving the encrypted blocks and their tags, the CSP selects a random $\tau \in Z_p$ and then produces $a_i = \pi_2(i, \tau)$, where $1 \leq i \leq n$. Then, the CSP verifies the correlation of the ciphertext blocks $\{c_i\}_{1 \leq i \leq n}$ and their tags $\{\sigma_i\}_{1 \leq i \leq n}$ through

$$e\left(\prod_{i=1}^{n} \sigma_i^{a_i}, g\right)$$

$$s = e\left(\prod_{i=1}^{n} H_1(h_e||i)^{a_i} \times \prod_{j=1}^{s} u_j^{\sum_{i=1}^{n} a_i c_{i,j}}, g^k\right). \quad (1)$$

If (1) holds, then the CSP stores the received ciphertext blocks, block tags, and file supplementary information. Otherwise, the CSP declines the storage request due to the mismatch between the uploaded ciphertext blocks and their tags.

- After storing the user data, the CSP first calculates the identity hash value of the data uploader as $h' = H_1(Ind_U)$, and then checks whether $e(h^k, g) = e(h', g^k)$ holds or not. If yes, the CSP publishes $h_e$ and $h^k$ to the smart contract $SC$. Otherwise, the CSP will report the failure and abort the storage service.
- After receiving $h_e$ and $h^k$, $SC$ first checks that they are from its deployer (i.e., the CSP). If yes, $SC$ then adds a mapping item with the key $h_e$ and the value $h^k$ to the $D$ on the blockchain.
- The initial user utilizes the local $h_e$ as the key to extract the value $h^k$ from the $D$. Then, the initial user checks whether the retrieved $h^k$ is correct. If not, the initial user reports the failure.

Note that it is easy to extend our scheme to support block-level deduplication through introducing two additional operations before generating the tags of the ciphertext blocks.

- For each ciphertext block $c_i$, the initial user computes its tag as $T_i = H_2(c_i)$, and then transmits the tag set $T = \{T_i\}_{1 \leq i \leq n}$ to CSP for detecting the duplicate data blocks.
- The CSP creates a non-duplicate block index set $I$ through comparing $T$ with its local tag set, which is returned to the initial user for determining which data blocks should be uploaded. In this way, the user only needs to upload the ciphertext blocks $c_i (i \in I)$ to the CSP, instead of the entire ciphertext $C$.

*Subsequent upload:* If $F$ is duplicated, the user performs the probabilistic PoW with the CSP as follows.

- The subsequent uploader firstly requests a challenge sequence $chal = \{c, \tau\}$ from the CSP, where $c$ is the number of the challenged blocks and $\tau$ is a random element in $Z_p$. Then, the subsequent user computes $c_{\pi_1(i,\tau)} = Enc(k_{\pi_1(i,\tau)}; m_{\pi_1(i,\tau)})$, where $k_{\pi_1(i,\tau)} = H_2(m_{\pi_1(i,\tau)})$ and $1 \leq i \leq c$. Finally, the subsequent user feeds back $g^k||h_\tau||h^k$ as the PoW to the CSP, where $h_\tau = H_2(\tau||\{c_{\pi_1(i,\tau)}\}_{1 \leq i \leq c})$ and $h^k = H_1(ID_U)^k$.
- After receiving the PoW, the CSP determines whether this subsequent user is the true owner of the file $F$ or not. There are three deterministic conditions to be met: i) the CSP is able to use $g^k$ as the index to locate its previously stored $C$ for the later computation; ii) $h'_\tau = h_\tau$ indicating that the subsequent user actually possesses the file $F$, where the CSP uses its stored ciphertext blocks to compute $h'_\tau = H_2(\tau||\{c'_{\pi_1(i,\tau)}\}_{1 \leq i \leq c})$; iii) $e(h^k, g) = e(h', g^k)$ indicating that the subsequent user with the identity $ID_U$ is interacting with the CSP, where the CSP computes $\tilde{h}' = H_1(ID_U)$ at local and retrieves $g^k$ with the help of $h^k$. If any of the above three conditions cannot be met, the CSP considers that this subsequent user is not the true owner of the file $F$ and terminates the subsequent storage service of this user. Otherwise, the CSP appends $h^k$ to the file supplementary information of $F$, and transmits $h_e$ and $h^k$ to the smart contract $SC$, where $h_e$ can be retrieved via $h^k$ by the CSP.
- After receiving $h_e$ and $h^k$ from the CSP, $SC$ appends $h^k$ to the value tuple corresponding to the key $h_e$ of $D$ on the blockchain.
- The subsequent user verifies the successful registration of his/her blinding identity on the blockchain by using $h_e$ as the index to query the mapping table $D$ and checking whether the corresponding entry contains $h^k$. If not, the subsequent user reports the failure.

Note that our scheme realizes the PoW in a probabilistic way, i.e., the CSP only samples a portion of user blocks for generating the PoW, which is the same as the previous schemes [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16] that realizes the integrity auditing through only challenging a portion of user blocks for producing the proof of data integrity. As illustrated in [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], when the number of sampled data blocks is large enough, the probabilistic integrity auditing algorithm still has a high probability of detecting data anomalies so that our hash-based PoW generated by probabilistic sampling can also have a high probability of finding out the inconsistent data blocks between the CSP and its users.

4) *Integrity Auditing:* The user randomly selects an element $r$ from $Z_p$, and computes the blinding file tag $g^{r\vartheta k}$, which is transmitted to the TPA together with $g^r$. Then, the TPA performs integrity auditing through the interactions between the blockchain and the CSP as follows. The pseudo-algorithm of IntegrityAuditing is provided in Algorithm 2 and the detailed procedure is given below.

- The TPA generates a challenge $chal = \{c, \tau\}$, where $c \in [1, n]$ and $\tau \in Z_p$ are both randomly selected. The TPA transmits the auditing sequence $(g^{r\vartheta k}, g^r, chal, ID_U)$ to the CSP, in which user identity $ID_U$ serves as an index to identify the user to be audited and $g^{r\vartheta k}$ acts as an index to further locate the specific audited file of that user.
- While obtaining $g^{r\vartheta k}, g^r, chal$ and $ID_U$, the CSP traverses each file belonging to the user with the identity $ID_U$ and finds out the audited file $F$ through checking

$$e(g^{r\vartheta k}, g) = e(g^{k'}, g^{r\vartheta}), \quad (2)$$

where $g^{k'}$ is the file tag maintained locally by the CSP.

**Algorithm 2:** Integrity Auditing.

**Input:** User identity $ID_U$, user block signing key $k$, system parameters $\{u_1, u_2, \ldots, u_s, \pi_1, \pi_2, e, g, g^{\vartheta}, H_1, H_2\}$.

1: User transmits an auditing request $(g^r, g^{r\vartheta k})$ to TPA, where $r$ is random in $Z_p$.
2: TPA computes a challenge $chal \leftarrow \{c, \tau\}$, where $c \in [1, n]$ and $\tau \in Z_p$ are both randomly selected.
3: TPA sends an auditing sequence $(g^r, g^{r\vartheta k}, chal, ID_U)$ to CSP.
4: CSP locates all the file $F$ associated with $ID_U$ and finds out the audited file among them through checking

$$e(g^{r\vartheta k}, g) = e(g^{k'}, g^{r\vartheta}).$$

5: CSP randomly selects a masking value $x \in \mathbb{Z}_p$.
6: CSP computes

$$\mu_j \leftarrow \sum_{i=1}^{c} (b_i + x)\, c_{a_i,j}, v \leftarrow \prod_{i=1}^{c} \sigma_{a_i}^{b_i+x}, \zeta \leftarrow \prod_{i=1}^{c} H_1(h_e \| a_i)^{b_i+x},$$

where $1 \le j \le s$, $a_i \leftarrow \pi_1(i, \tau)$, $b_i \leftarrow \pi_2(i, \tau)$ and $x$ is a random element in $Z_p$.
7: CSP transmits the integrity proof $proof \leftarrow \{\{\mu_1, \mu_2, \ldots, \mu_s\}, \nu, \zeta\}$ to TPA.
8: TPA verifies proof by checking

$$e(\nu, g^r) = e\left(\zeta \times \prod_{j=1}^{s} u_j^{\mu_j}, g^{h_e}\right).$$

9: **if** verification holds **then**
10: $q \leftarrow 1$
11: **else**
12: $q \leftarrow 0$
13: **end if**
14: TPA publishes $(g^{r\vartheta k}, g^r, chal, proof, q)$ on the blockchain for user verification.

- After that, the CSP first calculates $a_i = \pi_1(i, \tau)$ and $b_i = \pi_2(i, \tau)$, where $1 \le i \le c$. The CSP then randomly selects $x \in Z_p$ as a masking value, which is adopted in the generation of integrity proofs and is regenerated at each audit. The CSP computes $\mu_j = \sum_{i=1}^{c}(b_i + x)c_{a_i,j}$, $\nu = \prod_{i=1}^{c} \sigma_{a_i}^{\vartheta(b_i+x)}$ and $\zeta = \prod_{i=1}^{c} H_1(h_e||a_i)^{(b_i+x)}$, where $1 \le j \le s$. At last, the CSP generates the integrity proof of the ciphertext file $C$ as $proof = \{\{\mu_1, \mu_2, \ldots, \mu_s\}, \nu, \zeta\}$, which is transmitted to the TPA.
- After receiving the integrity proof, the TPA handles the integrity verification by checking the following equation:

$$e(\nu, g^r) = e\left(\zeta \times \prod_{j=1}^{s} u_j^{\mu_j}, g^{r\vartheta k}\right). \tag{3}$$

If the above equation holds, the TPA outputs the auditing result $q = 1$. Otherwise, $q = 0$. The TPA publishes ($g^{r\vartheta k}$, $g^r$, $chal$, $proof$, $q$) on the blockchain.

**Algorithm 3:** Data Retrieval.

**Input:** User file identifier, user identity $ID_U$

1: User utilizes the identifier of the targeted file $F$ to retrieve its block signing key $k$.
2: User computes the blinding user identity as $h^k \leftarrow H_1(ID_U)^k$.
3: User sends a file-downloading request with $h^k$ to CSP.
4: CSP verifies the ownership between $F$ and its requested user via

$$e(h^k, g) = e(h', g^k),$$

where $g^k$ is retrieved by taking $h^k$ as the lookup key.
5: **if** verification succeeds **then**
6: CSP feeds back the ciphertext $C$ and its encrypted keys $C_k$.
7: User decrypts $k_i \leftarrow Dec(k, C_k)$
8: **for** $i = 1$ to $n$ **do**
9: User recovers $m_i \leftarrow Dec(k_i, c_i)$
10: **end for**
11: **if** $k = H_2(m_1 \| m_2 \| \cdots \| m_n)$ **then**
12: User outputs the plaintext file $F = m_1 \| m_2 \| \cdots \| m_n$
13: **else**
14: User reports the failure.
15: **end if**
16: **else**
17: CSP rejects the downloading request.
18: **end if**

- The user is able to take $g^{r\vartheta k}$ as a public index to retrieve and verify his/her auditing result directly from the blockchain. For the sake of economic benefit, the TPA may perform dishonestly. For example, the TPA may directly release the auditing results without interacting with the CSP, and the TPA may repeatedly release the auditing results of the same blocks, etc. In order to detect the misbehaviors of the CSP, the user can check the on-chain auditing records from time to time.

5) *Data Retrieval:* When a data file $F$ is to be recovered, the user with the identity $ID_U$ should firstly check the integrity of its ciphertext file $C$. If it is verified as intact, the user is able to reconstruct the plaintext $F$ from the ciphertext $C$. The pseudo-algorithm of DataRetrieval is summarized in Algorithm 3 and the details are as follows.

- The user first utilizes the identifier of $F$ to find out its block signing key $k$. Then, the user computes $h^k$, and transmits a file-downloading request together with $h^k$ to the CSP.
- While obtaining the file-downloading request, the CSP firstly uses $h^k$ as the index to retrieve $g^k$. Then, the CSP checks the ownership relationship between $F$ and its requested user through judging whether $e(h^k, g) = e(h', g^k)$ holds or not, where $h' = H_1(ID_U)$ is computed locally. If yes, the CSP uses $h^k$ as the index to retrieve the requested ciphertext $C = c_1||c_2|| \cdots ||c_n$ and the encrypted block keys $Ck$, which are fed back to the requested user. Otherwise, the CSP rejects the downloading request.

- The user utilizes the key $k$ to compute the block keys as $\{k_1, k_2, \ldots, k_n\} = Dec(k, Ck)$, where $Dec$ is the decryption algorithm corresponding to the chosen symmetric encryption algorithm $Enc$.
- The user decrypts the ciphertext $C = \{c_i\}_{1 \le i \le n}$ into the plaintext $F = m_1 || m_2 || \cdots || m_n$ by computing $m_i = Dec(k, c_i)$, where $1 \le i \le n$.
- The user validates the integrity of the decrypted $F$ through detecting whether $k = H_2(F)$ holds. If yes, the user accepts it. Otherwise, the user rejects it and reports failure.

## V. Further Discussion

It is worth noting that HCE is indeed vulnerable to plaintext prediction attacks, especially when an outsourced block encrypted by HCE only has a few bits. However, in the application scenarios of cloud storage, the outsourced data blocks are typically large in size; otherwise, outsourcing such data would be unnecessary. In this way, breaking an outsourced block by plaintext prediction attack becomes computationally infeasible. For example, even if a supercomputer is capable of testing a billion billion ($10^{18}$) bits per second, an exhaustive search of a 128-bit block size would still require on the order of $10^{12}$ years.

On the other hand, to completely address this problem, we can simply introduce key servers $KS$s to blind the block signing key (i.e., block hash value), following the approach in [33], [35]. The basic idea is to send the block hash value $H(m)$ to a set of $K$ key servers $KS$s. Each $KS_i$ independently computes a blinded component $H(m)^{\gamma_i}$ using its own secret share $\gamma_i$. The user then combines these partial results to derive the final block encryption key $H(m)^{\gamma}$, where $\gamma$ is implicitly defined by the aggregation of all individual shares $\{\gamma_i\}_{1 \le i \le K}$ and at no point does any single KS learn the complete value of $\gamma$. In such a way, even for low-entropy data, our block signing keys can still be prevented from being deduced under plaintext prediction attacks. We refer interested readers to [33], [35] for a detailed description, as this component is orthogonal to the main contribution of this paper and is omitted here.

## VI. Theoretical Analysis

This section analyzes the correctness and security of the proposed protocol.

### A. Correctness

*1) Correctness of Deduplication:* Correctness of deduplication relies on the collision resistance of the hash function.

*Proof:* Assume one user $U$ has uploaded the ciphertext $C$ and its hash value $h_e$ to the CSP, where $h_e$ is then published on the blockchain by the CSP. When the user $U^*$ intends to upload the ciphertext $C^*$, it first computes its hash value $h_e^*$ and then checks whether $h_e^*$ has been published on the blockchain.

It is obvious that $h_e = h_e^*$ only in the case of $C = C^*$, i.e., $U$ and $U^*$ have the identical ciphertext file. As a result, with the help of on-chain logs, users can detect duplicate files through the hash values of their ciphertexts. □

*2) Correctness of Auditing:* Correctness of auditing follows from the bilinearity of the pairing.

*Proof:* Suppose a user entrusts the TPA to audit a ciphertext file $C$ through providing the blinding file tag $g^{r\vartheta k}$ and the related blinding parameter $g^r$. The TPA checks the integrity proof $proof = \{\{\mu_1, \mu_2, \ldots, \mu_s\}, \nu, \zeta\}$ by using (3). The TPA calculates both sides of (3) as follows:

$$\begin{aligned} e(\nu, g^r) &= e\left(\prod_{i=1}^{c} \sigma_{a_i}^{\vartheta(b_i+x)}, g^r\right) \\ &= e\left(\prod_{i=1}^{c}\left(\left(H_1(h_e||a_i)\prod_{j=1}^{s} u_j^{c_{a_i,j}}\right)^k\right)^{b_i}, g^{r\vartheta}\right) \\ &\quad \times e\left(\prod_{i=1}^{c}\left(\left(H_1(h_e||a_i)\prod_{j=1}^{s} u_j^{c_{a_i,j}}\right)^k\right)^{x}, g^{r\vartheta}\right) \\ &= e\left(\prod_{i=1}^{c} H_1(h_e||a_i)^{b_i}\prod_{j=1}^{s} u_j^{\sum_{i=1}^{c} b_i c_{a_i,j}}, g^{r\vartheta k}\right) \\ &\quad \times e\left(\prod_{i=1}^{c}\left(H_1(h_e||a_i)\prod_{j=1}^{s} u_j^{c_{a_i,j}}\right)^x, g^{r\vartheta k}\right) \end{aligned} \tag{4}$$

and

$$\begin{aligned} & e\left(\zeta \times \prod_{j=1}^{s} u_j^{\mu_j}, g^{r\vartheta k}\right) \\ &= e\left(\prod_{i=1}^{c} H_1(h_e||a_i)^{b_i+x} \times \prod_{j=1}^{s} u_j^{\sum_{i=1}^{c}(b_i+x)c_{a_i,j}}, g^{r\vartheta k}\right) \\ &= e\left(\prod_{i=1}^{c} H_1(h_e||a_i)^{b_i} \times \prod_{j=1}^{s} u_j^{\sum_{i=1}^{c} b_i c_{a_i,j}}, g^{r\vartheta k}\right) \\ &\quad \times e\left(\prod_{i=1}^{c}\left(H_1(h_e||a_i)\prod_{j=1}^{s} u_j^{c_{a_i,j}}\right)^x, g^{r\vartheta k}\right) \end{aligned} \tag{5}$$

It can be easily observed that $e(\nu, g^r) = e(\zeta \times \prod_{j=1}^{s} u_j^{\mu_j}, g^{r\vartheta k})$, which proves the correctness of the verifying (3). □

### B. Security Analysis

We analyze the security of our scheme from the perspectives of data privacy, unforgeability, OP protection, and resistance to DFA and DPD.

*1) Data Privacy:* Our solution is designed to safeguard the privacy of user data, meaning that only the data owners are able to access the plaintext file $F = \{m_i\}_{1 \le i \le n}$, while other entities are unable to access this plaintext file $F$.

*Theorem VI.1:* The BTAD can achieve data privacy if the underlying AES-256 is semantically secure.

*Proof:* The proof of data privacy is achieved by reducing it to the semantic security of the underlying symmetric encryption

scheme. Specifically, in our BTAD, we employ the well-known AES-256 to perform symmetric encryption on each block in $F$ with the encryption keys $\{sk_i\}_{1\le i\le n}$, respectively. Since AES-256 is semantically secure, the adversary is only able to reconstruct $F$ through cryptanalyzing the confidential $\{sk_i\}_{1\le i\le n}$. On the one hand, the block encryption keys $\{sk_i\}_{1\le i\le n}$ might be cracked through their ciphertext $Ck$ stored on the CSP. On the other hand, the CSP possesses more knowledge compared to other entities. Thus, we only need to demonstrate that data privacy can be protected against the CSP as the adversary. Considering that the ciphertext $Ck$ is also computed using the semantically secure AES-256, the CSP cannot reconstruct the block encryption keys $\{sk_i\}_{1\le i\le n}$ from the $Ck$. Consequently, our BTAD can achieve data privacy. □

*2) Unforgeability:* Our scheme can realize the property of unforgeability, indicating that the CSP cannot utilize the forged user data to generate the valid integrity proof that can pass the verification of the TPA.

*Theorem VI.2:* The BTAD can achieve unforgeability if the DLP is hard to solve in probabilistic polynomial time.

*Proof:* The key idea of the proof is to reduce unforgeability to the hardness of the discrete logarithm problem. On the one hand, the CSP is able to cheat the TPA by falsifying ciphertext blocks and their tags if it gains the secret block signing key for generating the block tags. In our scheme, the CSP is allowed to have the file tag $g^k$, the blinding user identity $\tilde{h}^k$, the blinding file tag $g^{r\vartheta k}$ and $g^r$. Although $g$ is public to the CSP, the hardness of solving $k$ from $(g^k, \tilde{h}^k, g^{r\vartheta k}, g^r)$ is equivalent to addressing the DLP. Therefore, any probabilistic polynomial-time adversary can hardly crack $k$. This also means that the CSP is unable to derive the block signing key $k$ for generating block tags.

On the other hand, the CSP may use an incorrect block signing key $k'$ to falsify block tags $\{\sigma'_i\}_{1\le i\le n} = \{(H_1(h'_e||i)\prod_{j=1}^{s} u_j^{c'_{i,j}})^{k'}\}_{1\le i\le n}$ for the counterfeit ciphertext $C' = c'_1||c'_2||\cdots||c'_n$, where $c'_i = \{c'_{i,j}\}_{1\le j\le s}$. Then, the CSP can generate the integrity proof $proof = \{\{\mu'_j\}_{1\le j\le s}, \nu', \zeta'\}$, where $\mu'_j = \sum_{i=1}^{c}(b_i + x)c'_{a_i,j}$, $\nu' = \prod_{i=1}^{c}\sigma'^{\vartheta(b_i+x)}_{a_i}$ and $\zeta' = \prod_{i=1}^{c} H_1(h'_e||a_i)^{(b_i+x)}$. While obtaining the integrity proof, the TPA handles integrity verification using (3). In this case, the left side of (3) is computed as

$$e(\nu', g^r) = e\left(\prod_{i=1}^{c} H_1(h'_e||a_i)^{b_i} \prod_{j=1}^{s} u_j^{\sum_{i=1}^{c} b_i c'_{a_i,j}} \times \prod_{i=1}^{c}\left(H_1(h'_e||a_i)\prod_{j=1}^{s} u_j^{c'_{a_i,j}}\right)^{x}, g^{r\vartheta k'}\right), \quad (6)$$

and the right side of (3) is computed as

$$e\left(\zeta' \times \prod_{j=1}^{s}(u'_j)^{\mu_j}, g^{r\vartheta k}\right)$$

$$= e\left(\prod_{i=1}^{c} H_1(h'_e||a_i)^{b_i} \prod_{j=1}^{s} u_j^{\sum_{i=1}^{c} b_i c'_{a_i,j}} \times \prod_{i=1}^{c}\left(H_1(h'_e||a_i)\prod_{j=1}^{s} u_j^{c'_{a_i,j}}\right)^{x}, g^{r\vartheta k}\right). \quad (7)$$

Because the $k'$ in (6) is not equal to the $k$ in (7), the integrity verifying equation (3) does not hold. This implies that the CSP is unable to cheat the TPA with the incorrect block signing key. Consequently, we can conclude that our BTAD realizes the property of unforgeability. □

*3) OP Protection:* The adversary $\mathcal{A}$ might eavesdrop on the OP from the auditing logs of the TPA and the deduplication pattern on the blockchain. Based on this, we discuss how our BTAD can effectively protect the OP, which is achieved by reducing OP protection to the randomness of the blinding parameters.

Firstly, we demonstrate that $\mathcal{A}$ cannot obtain the OP from the auditing logs of the TPA.

- Because our scheme uses blinding file tags during the delegation of the auditing tasks, it is evident that adversary $\mathcal{A}$ cannot accurately determine which files are owned by an identical user. Next, we primarily demonstrate that adversary $\mathcal{A}$ cannot gain knowledge about which users have the same file.
- Assume two users $U$ and $U'$ upload the same ciphertext $C$ to the CSP and entrust the TPA to perform auditing tasks against the same file. We then demonstrate that even if the auditing requests during the two audit processes are the same, they are still unable to reveal whether $U$ and $U'$ possess the same file, where an auditing request is composed of the blinding file tag, the blinding parameter, the challenge sequence and the user identity.
  - For the blinding file tags and the blinding parameters in the auditing request, it is difficult for a PPT adversary to distinguish $(g^{r\vartheta k}, g^r)$ from $(g^{r'\vartheta k}, g^{r'})$, where $r$ and $r'$ are randomly selected in $Z_p$ by $U$ and $U'$, respectively. Furthermore, without the secret $\vartheta$ of the CSP, it is also difficult for a PPT adversary to detect whether $(g^{r\vartheta k}, g^r)$ and $(g^{r'\vartheta k}, g^{r'})$ are associated with the same file. Thereafter, whether $U$ and $U'$ have the same file is not disclosed from the blinding file tags and the blinding parameters.
  - For the challenge sequence and the user identity in the auditing request, it is obvious that they do not leak the OP. The reason is that their generation has nothing to do with user files.
  - For the integrity proof, the CSP produces unique proofs for distinct users by employing different masking values. The masking values $x$ and $x'$ are randomly selected by the CSP for users $U$ and $U'$, respectively. On the one hand, recovering $x$ from the proof components $\nu$ and $\zeta$ is infeasible under the hardness of the underlying discrete logarithm problem, while deriving $x$ from the remaining proof components $\{\mu_j\}_{1\le j\le s}$ is also infeasible due to the underdetermined nature of the extracted equations.

Besides, $x$ is not exposed as an explicit multiplier in the proof, which prevents the attack in [34] based on the greatest common divisor. Thus, $x$ is computationally hidden in the integrity proof, and the same observation also holds for $x'$ due to the same reason. On the other hand, since the $x$ in $\mu_j$ and the $x'$ in $\mu'_j$ are different, $\mu_j$ is not equal to $\mu'_j$ for each $1 \leq j \leq s$. Due to the difference of $x$ in $\nu$ and $x'$ in $\nu'$, $\nu$ is not equal to $\nu'$. Additionally, because $x$ in $\zeta$ and $x'$ in $\zeta'$ are different, $\zeta$ is not equal to $\zeta'$. Therefore, all users and the TPA are unable to discern whether $U$ and $U'$ have the same file using the integrity proofs $\{\{\mu_1, \mu_2, \ldots, \mu_s\}, \nu, \zeta\}$ and $\{\{\mu'_1, \mu'_2, \ldots, \mu'_s\}, \nu', \zeta'\}$.

- For the auditing result, it clearly does not leak OP, because it is only a boolean value for any file.

Secondly, we demonstrate that the adversary $\mathcal{A}$ cannot obtain the OP from the on-chain deduplication pattern, which is built on a mapping table. For each mapping item, its key is the hash value of ciphertext file, and its value is the blinding user identities.

- The adversary $\mathcal{A}$ cannot decide which users have the same file due to the secrecy of block signing key. Specifically, it is almost impossible to find out $ID_U$ from the blinding user identity $h^k = H_1(ID_U)^k$ without the knowledge of the block signing key $k$. As a consequence, the adversary $\mathcal{A}$ cannot determine which users have the same file.
- Besides, the adversary $\mathcal{A}$ also cannot learn which files are owned by an identical user since different files have different file hash values.

Based on the above discussion, our BTAD can realize OP protection.

*4) Resistance to DFA:* One objective of our work is to protect the subsequent user from acquiring forged copies.

- During the case of initial uploading, our scheme mandates the CSP to verify the correspondence of the ciphertext blocks and their tags from the initial user. Therefore, in order to bypass this CSP verification, the initial user is limited to outsource the forged ciphertext $C'$ along with its tag set $T'$ so that the poisoned ciphertext is stored on the CSP.
- During the case of subsequent uploading, our scheme requires the CSP to verify the file ownership of the subsequent user by checking the probabilistic PoW, so as to resist the DFA launched by the initial uploader. Thanks to the probabilistic PoW, the CSP does not need to verify all ciphertext blocks. Instead, it samples only a subset of ciphertext blocks according to the challenge $\tau$ and then checks whether the sampled blocks stored by the CSP are consistent with those possessed by the subsequent uploader. Specifically, the CSP compares its locally computed $h'_\tau = H_2(\tau||\{c'_{\pi_1(i,\tau)}\}_{1\leq i\leq c})$ with the feedback $h_\tau = H_2(\tau||\{c_{\pi_1(i,\tau)}\}_{1\leq i\leq c})$ in the PoW from the subsequent uploader. Considering that the ciphertext file $C' = \{c'_i\}_{1\leq i\leq n}$ on the CSP is maliciously forged due to the DFA launched by the initial uploader, it is inconsistent with the duplicate file $C = \{c_i\}_{1\leq i\leq n}$ possessed by the subsequent uploader. In such way, the probability $P_d$ of the CSP detecting the forged data can be computed as

$$P_d = 1 - \prod_{i=0}^{c-1} \frac{n-\kappa-i}{n-i} \tag{8}$$

where $\kappa$ represents the number of forged data blocks in $C'$. It can be directly observed that $P_d$ approaches one when $c$ is large enough. For example, for a file with $n = 10000$ blocks, if 1% of the blocks are forged, then $P_d \geq 99\%$ by challenging only 4.6% of ciphertext blocks. This implies that in front of the DFA, there is a high probability that the CSP will have $h_\tau \neq h'_\tau$ and thus consider $C$ as a new file. After that, the probabilistic PoW can efficiently disable the DFA initiated by the initial uploader, without requiring full-file ownership verification or re-computation over all ciphertext blocks.

Based on the above description, the resistance to the DFA depends on the corresponding relationship of ciphertext blocks and their tags outsourced by the initial user and the PoW. Next, we will proceed to demonstrate these two operations, both of which rely on the equality of the underlying ciphertext blocks.

*Corresponding relationship of ciphertext blocks and their tags:* Assuming the initial user uploads a falsified ciphertext $C = \{c_i\}_{1\leq i\leq n}$, while the tag set $T$ of the actual ciphertext $C' = \{c'_i\}_{1\leq i\leq n}$ is computed as $T = \{\sigma_i = (H_1(h_e||i)\prod_{j=1}^{s} u_j^{c'_{i,j}})^k\}_{1\leq i\leq n}$. Next, the CSP verifies the correspondence of the falsified ciphertext blocks and their tags through (1). Specifically, the left side of (1) is computed as

$$\begin{aligned} e\left(\prod_{i=1}^{n}\sigma_i^{a_i}, g\right) &= e\left(\prod_{i=1}^{n}((H_1(h_e||i)\prod_{j=1}^{s}u_j^{c'_{i,j}})^k)^{a_i}, g\right) \\ &= e\left(\prod_{i=1}^{n}H_1(h_e||i)^{a_i} \times \prod_{j=1}^{s}u_j^{\sum_{i=1}^{n}a_i c'_{i,j}}, g^k\right). \end{aligned} \tag{9}$$

However, the right side (1) is computed as $e(\prod_{i=1}^{n} H_1(h_e||i)^{a_i} \times \prod_{i=1}^{n} u_j^{\sum_{j=1}^{s} a_i c_{i,j}}, g^k)$. Clearly, the condition that (1) becomes true is only in the case of $c_i = c'_i$, where $1 \leq i \leq n$. As a result, the CSP is able to verify the corresponding relationship of ciphertext blocks and their tags.

*PoW:* When the subsequent user intends to upload a file that already exists, the CSP is required to verify the PoW by checking whether $h'_\tau = h_\tau$ holds or not, where $h'_\tau = H_2(\tau||\{c'_{\pi_1(i,\tau)}\}_{1\leq i\leq c})$ and $h_\tau = H_2(\tau||\{c_{\pi_1(i,\tau)}\}_{1\leq i\leq c})$. Obviously, $h'_\alpha = h_\alpha$ holds only in the case of $\{c'_{\pi_1(i,\tau)}\}_{1\leq i\leq c} = \{c_{\pi_1(i,\tau)}\}_{1\leq i\leq c}$. Consequently, the correctness of the PoW is guaranteed.

Because the corresponding relationship between ciphertext blocks and their tags can be checked by the CSP and the ciphertext consistency between the initial user and the subsequent user can be verified through PoW, it can be concluded that the subsequent user will not be affected by the DFA.

*5) Resistant to DPD:* On the one hand, for the sake of economic profits, the CSP may perform the DPD by not appending the blinding identities of the initial/subsequent users to the

TABLE III
EXPERIMENTAL NOTATIONS

| Symbol | Description |
|---|---|
| $n_f$ | Number of unique files stored at the CSP |
| $n_u$ | Number of users possessing the same file |
| $n_c$ | Number of integrity auditing operations |
| $c_h$ | Hash computation on a small amount of data |
| $c_{h'}$ | Hash computation on a large-scale user file |
| $c_e$ | One exponentiation in $G_1$ |
| $c_a$ | One addition in $\mathbb{Z}_p$ |
| $c_m$ | One multiplication in $\mathbb{Z}_p$ |
| $c_{m'}$ | One multiplication in $G_1$ |
| $c_{enc}$ | One AES-256 encryption |
| $c_{dec}$ | One AES-256 decryption |

on-chain mapping table. In such a way, the CSP can gain an extra profit by making the subsequent user pay the storage fee without the deserved discount.

To meet this challenge, our countermeasure is that after uploading the file to the CSP, the user has to check whether his/her blinding user identity is added to the on-chain mapping table. Note that the user has a strong incentive to do this to gain a lower discount on the cloud storage fee. Thereafter, our BTAD can protect the subsequent user from being affected by the DPD.

## VII. PERFORMANCE EVALUATION

The performance of our BTAD is evaluated through theoretical analysis and experimental results. What is more, we provide a detailed comparison between our BTAD and the existing state-of-the-art schemes [33], [35]. The reason why we choose these two schemes as the main performance comparison benchmarks because they are the most relevant state-of-the-art blockchain-based integrated schemes to our work. Specifically, [33] is closely related to the protection of ownership privacy (OP), while [35] is closely related to the resistance to deduplication pattern degradation (DPD). Since our BTAD mainly focuses on simultaneously protecting OP and resisting DPD, comparing with [33] and [35] can directly reflect the performance overhead introduced by these two core mechanisms.

### A. Theoretical Analysis

We conduct the theoretical analysis in terms of the storage, communication, and computation costs. For the sake of simplicity, we make the following two simplifications during the analysis: one is that we only focus on the highest order of the cost with the largest coefficient, which certainly accounts for the largest proportion of the whole cost; the other is that each data file is assumed to have the same number of blocks and be owned by the same number of users. In addition, we do not consider introducing multiple server key servers to generate user keys by using the secret key sharing technique, since it can be directly adopted in an arbitrary protocol of secure deduplication and integrity auditing. The experimental notations are summarized in Table III.

*Functionality and Security:* Our BTAD is able to enable users to upload encrypted data to the CSP and check the integrity of their outsourced data. In the phase of data auditing, users can remain offline, and only the initial user generates and uploads data block tags, while subsequent users who have an identical file do not. The CSP is able to achieve transparent deduplication of encrypted blocks and their tags. Furthermore, the proposed BTAD can also protect OP, resist DFA and DPD. On the other hand, the protocol of [33] solely relies on the CSP for checking whether a user is initial or subsequent and thus cannot resist the threat of DPD. The protocol of [35] mandates users to remain online to audit the integrity of their uploaded data due to no TPA involved, and do not consider how to deal with OP leakage. Besides, the protocol of [35] does not support block-level deduplication since its ciphertext blocks are generated by chunking the whole ciphertext file, instead of using the HCE algorithm. Our BTAD is the first one to comprehensively consider the above-mentioned issues and completely address them.

*Storage cost:* It is composed of on-chain and off-chain parts. In our scheme, the on-chain storage cost of is incurred by the auditing information and the mapping table, which is about $(\sum_{i=1}^{n_f} n_i + n_c)|G_1| + n_c s|Z_p|$ bits. The off-chain storage cost of our scheme is mainly determined by cloud storage and thus is about $|C| + n|G_1| + n_f n|Z_p|$ bits, where cloud storage includes the ciphertext blocks, their tags and file supplementary information.

From Table IV, it can be observed that the on-chain storage costs of these three schemes have their advantages and disadvantages under different parameter configurations. In addition, our BTAD has a relatively high off-chain storage cost, which is mainly due to the need to achieve the functionality of data retrieval. Specifically, the ciphertext of each file block encryption key is stored on the CSP in our BTAD, while it is stored on the blockchain in the protocol of [33]. And, the functionality of data retrieval is not considered in the protocol of [35].

*Communication cost:* It is mainly generated by the procedures of initial uploading, subsequent uploading, integrity auditing and data retrieval. To be specific, for initial uploading, the communication cost is mainly determined by the ciphertext file, its block tags and supplementary information, which is about $|C| + n|G_1| + n|Z_p|$ bits. For subsequent uploading, the communication cost is mainly caused by the PoW, which is about $3|G_1| + 2|Z_p|$ bits. For integrity auditing, the communication cost is mainly incurred by the auditing request and the integrity proof, which is about $7|G_1| + 2(s+1)|Z_p| + 2|c| + |ID_U|$ bits. For the data retrieval, the communication cost is mainly introduced by the data downloading, which is about $|C| + |G_1| + n|Z_p|$ bits.

From Table V, it can be found that: i) our BTAD slightly increases the communication costs of the initial uploading and the subsequent uploading. This is because our BTAD introduces some extra communication costs in order to gain stronger security against the DPD; ii) our BTAD has the lowest communication cost of integrity auditing due to the minimal information interaction between the auditor and the blockchain; iii) our BTAD has almost the same communication cost as the protocol of [33] in the phase of data retrieval, while the protocol of [35] does not consider how to realize data retrieval.

TABLE IV
THE COMPARISON OF STORAGE COSTS AMONG DIFFERENT PROTOCOLS

| Protocol | On-chain | Off-chain |
|---|---|---|
| Song et al. [33] | $n_c\|G_1\| + (n_f n + n_c c)\|Z_p\|$ | $n_f\|C\| + n_f n\|G_1\| + n_f\|Z_p\|$ |
| Li et al. [35] | $n_c s\|G_1\| + n_c(s + log_2 n_u)\|Z_p\|$ | $n_f\|C\| + n_f n\|G_1\| + n_f\|Z_p\| + n_f n_u\|ID_U\|$ |
| The proposed BTAD | $(n_f n_u + n_c)\|G_1\| + n_c s\|Z_p\|$ | $n_f\|C\| + n_f n\|G_1\| + n_f n\|Z_p\|$ |

TABLE V
THE COMPARISON OF COMMUNICATION COSTS AMONG DIFFERENT PROTOCOLS

| Protocol | Initial uploading | Subsequent uploading | Integrity auditing | Data retrieval |
|---|---|---|---|---|
| Song et al. [33] | $\|C\| + (n+2)\|G_1\| + (n+2)\|Z_p\|$ | $3\|G_1\| + 5\|Z_p\| + \|c\|$ | $10\|G_1\| + 2(s+2)\|Z_p\| + 2\|c\| + 2\|ID_U\|$ | $\|C\| + \|G_1\| + n\|Z_p\|$ |
| Li et al. [35] | $\|C\| + n\|G_1\| + (n+4)\|Z_p\|$ | $5\|Z_p\|$ | $2s\|G_1\| + 2(s + 2log_2 n_u)\|Z_p\|$ | – |
| The proposed BTAD | $\|C\| + (n_u + n + 3)\|G_1\| + (n_f + n + 2)\|Z_p\|$ | $(n_u + 2)\|G_1\| + (n_f + 2)\|Z_p\|$ | $7\|G_1\| + 2(s+1)\|Z_p\| + 2\|c\| + \|ID_U\|$ | $\|C\| + \|G_1\| + n\|Z_p\|$ |

TABLE VI
THE COMPARISON OF COMPUTATION COSTS AMONG DIFFERENT PROTOCOLS

| Protocol | Initial uploading | Subsequent uploading | Integrity auditing | Data retrieval |
|---|---|---|---|---|
| Song et al. [33] | $nsc_e + nsc_{m'} + nc_m + nc_a + 3nc_h + 2nc_{enc}$ | $2cc_e + 2cc_{m'} + sc_m + 2sc_a + cc_{enc} + cc_h$ | $(2c+s)c_e + (2c+s)c_{m'} + csc_m + csc_a + cc_h$ | $sc_e + sc_{m'} + nc_{dec}$ |
| Li et al. [35] | $nsc_e + nsc_{m'} + nc_m + nc_a + 3nc_h$ | $3c_{h'}$ | $(2c+s)c_e + (2c+s)c_{m'} + csc_m + csc_a + cc_h$ | – |
| The proposed BTAD | $nsc_e + nsc_{m'} + nc_m + nc_a + 3nc_h + 2nc_{enc}$ | $cc_{enc} + cc_h$ | $(2c+s)c_e + (2c+s)c_{m'} + csc_m + csc_a + cc_h$ | $nc_{dec}$ |

*Computation cost:* It is mainly produced by the processes of initial uploading, subsequent uploading, integrity auditing and data retrieval. For initial uploading, its computation cost is mainly caused by the generation of ciphertext block tags and the integrity verification of the outsourced data, which is about $nsc_e + nsc_{m'} + nc_m + nc_a + 3nc_h + 2nc_{enc}$. For subsequent uploading, its computation cost is mainly generated by the PoW, which is about $cc_{enc} + cc_h$. For integrity auditing, its computation cost is mainly produced by proof generation and integrity verification, which is about $(2c+s)c_e + (2c+s)c_{m'} + csc_m + csc_a + cc_h$. For data retrieval, its computation cost is mainly incurred by data decryption, which is about $nc_{dec}$.

From Table VI, we can observe that our BTAD and the protocol [33] have a slightly higher computation cost than the protocol of [35] during the process of initial uploading. The reason is that the protocol of [35] does not consider how to support block-level deduplication, and generates the ciphertext blocks by partitioning the whole ciphertext file, while the other two protocols generate the ciphertext blocks by encrypting each partitioned data block in turn. For the phase of subsequent uploading, our BTAD has a lower computation cost than the other two protocols. The reason is that the protocol of [33] involves complex computation over an elliptic curve during the PoW and the protocol of [35] involves complex hash computation on the whole large-scale user file. In the phase of integrity auditing, these three protocols have similar computation costs due to their similar operations. During the phase of data retrieval, our BTAD has a much lower computation cost compared with the protocol of [33]. The reason is that our BTAD can directly use the blinding user identity to locate the file to be recovered without any additional operation, while the protocol of [33] locates the corresponding file by checking whether a random file tag satisfies a computationally-complicated equation over bilinear mapping.

It can also be observed that the system overhead scales linearly with the number of users and files, since each user and each file is processed independently in terms of storage, communication, and computation costs. For duplicated ciphertext files, subsequent users only incur lightweight PoW and verification costs, ensuring that the overall system overhead grows sublinearly with

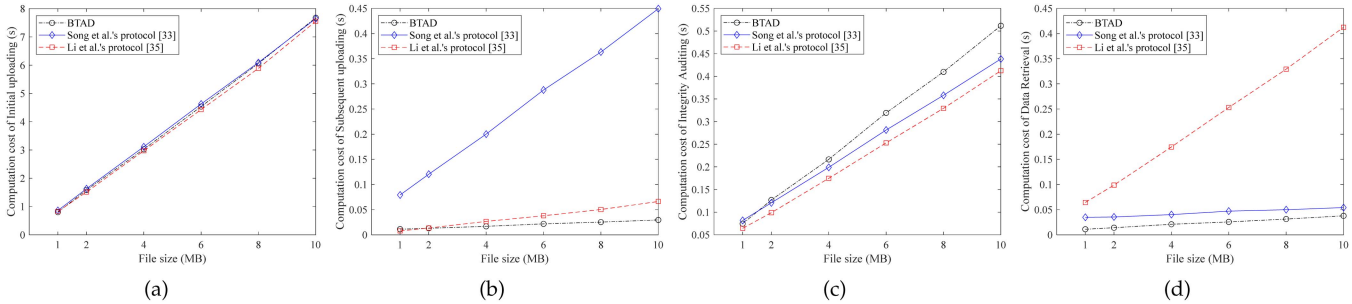


Fig. 2. Comparisons of off-chain computation cost under different file sizes for different schemes: (a) Computation cost of initial uploading. (b) Computation cost of subsequent uploading. (c) Computation cost of integrity auditing. (d) Computation cost of data retrieval.

respect to the number of replicas. Moreover, under frequent auditing operations, the auditing protocol will be executed repeatedly, and the overall overhead increases linearly with the number of audits.

### B. Experimental Results

We adopt a laptop running Windows 10 with an Intel i5-7200 U CPU and 8 GB RAM to carry out the experiments. We use the JAVA programming language to complete the off-chain procedures with the help of Java Pairing-Based Cryptography Library (JPBC) [40]. JDK 1.8 is adopted as the runtime and the version of the JPBC library is 2.0.0. We record the on-chain data on Sepolia, which is an alternative testing blockchain for Ethereum, and estimate the on-chain operations by using the gas consumption. The smart contract is implemented using the Solidity programming language and compiled in Remix IDE with Solidity compiler.

In our setting, we determine the bilinear pairing using the elliptic curve with the 160-bit security level and the chosen symmetric key encryption algorithm is AES-256. Specifically, we employ Type-A pairing with a 512-bit base field order and a 160-bit group order, where the size of each element in $Z_p$ and $G_1$ is 20 bytes and 128 bytes, respectively. We randomly generate the files with sizes ranging from 1 MB to 10 MB. Each file is divided into fixed-size blocks, where each block has 8 sectors of size 4 KB. Also, we suppose that during each deduplication checking and integrity auditing, 1/3 data blocks are challenged. Besides, each experimental result is an average of 100 times.

It should be noted that Sepolia is adopted only as a reproducible experimental testbed, rather than the only possible deployment platform. In practical cloud storage applications, a permissioned or consortium blockchain may be more suitable because it can provide lower transaction cost, higher throughput, and controllable participants. However, the performance of consortium blockchains is highly dependent on specific platforms, consensus protocols, node configurations, and hardware environments, and some consortium blockchains do not follow the same gas-based cost model as Ethereum. Therefore,we use Sepolia to evaluate the gas overhead of the Solidity smart contract in a standard EVM-compatible environment, while considering consortium-chain deployment as a practical optimization direction for real-world systems.

*1) Off-Chain Computation Cost:* As shown in Fig. 2, the off-chain computation cost is evaluated from the perspectives of initial uploading, subsequent uploading, integrity auditing and data retrieval, respectively, where the file size ranges from 1 MB to 10 MB. The computation costs of all three schemes increase with the file size. This is because, under a fixed-size block partitioning strategy, a larger file size leads to a larger number of data blocks, and the computation cost of the initial uploading phase grows linearly with the block number $n$. Fig. 2(a) depicts the computation cost of initial uploading operations, consisting of user key generation, file block encryption and block tag computation. It can be observed that the computation cost of our protocol is close to that of the protocol in [33], and is a little higher than that of the protocol in [35]. Please refer to Table IV for a specific reason, which is omitted here. Fig. 2(b) depicts the computation cost of subsequent uploading operations, which is mainly determined by the PoW. It can be found that our BTAD always has a lower computation cost than the protocols of [33] and [35] across different file sizes, which are also as expected in Table VI. Fig. 2(c) shows the computation cost of integrity auditing, which is composed of challenge generation, proof computation and integrity verification. We can observe that the computation cost of our BTAD is slightly higher than the protocols of [33] and [35], which is not consistent with the theoretical results in Table VI. This is because in order to truly protect the OP of the users, our BTAD introduces a random parameter to mask the integrity proof, which, however, enlarges the exponents in exponential operations over an elliptic curve during the generation of integrity proof, increasing the computation cost of our BTAD. Fig. 2(d) draws the computation cost of data retrieving operations, containing block decryption and correctness verification. We can find that our BTAD has a much lower computation cost than the protocol in [33], and the computation cost difference increases as the size of user file blocks increases, which coincides with the theoretical results in Table VI.

As shown in Fig. 3, we evaluate the off-chain computation costs of initial uploading, subsequent uploading, integrity auditing, and data retrieval when the user number increases from 10 to 100 with the file size fixed at 10 MB. The experimental results show that, as the number of users increases, the computation costs of all schemes generally increase. For initial uploading, our BTAD has a computation cost close to

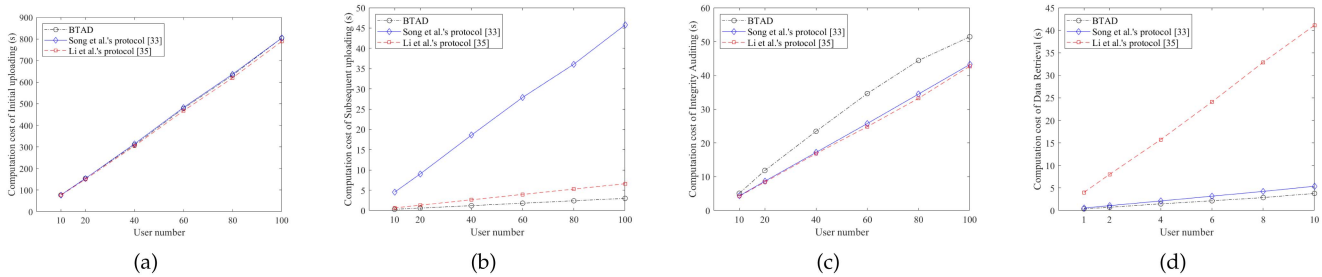


Fig. 3. Comparisons of off-chain computation cost under different user numbers for different schemes: (a) Computation cost of initial uploading. (b) Computation cost of subsequent uploading. (c) Computation cost of integrity auditing. (d) Computation cost of data retrieval.

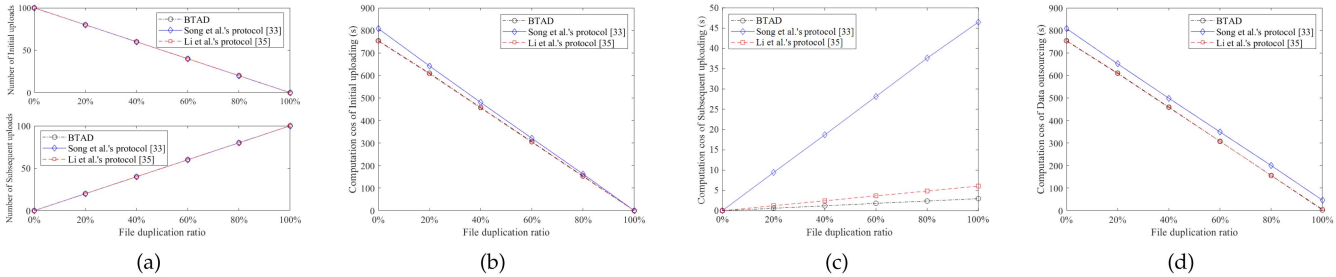


Fig. 4. Performance of data outsource under different file duplication ratios for different schemes: (a) Number of initial uploads and subsequent uploads. (b) Computation cost of initial uploading. (c) Computation cost of subsequent uploading. (d) Computation cost of data outsourcing.

the compared schemes, since the initial uploader still needs to perform file encryption and tag generation. For subsequent uploading, our BTAD achieves much lower computation cost than the other two schemes, because the subsequent uploader only needs to complete a lightweight probabilistic PoW. For integrity auditing, although BTAD introduces random masking to protect ownership privacy, its computation cost grows steadily with the user number and remains acceptable. For data retrieval, BTAD also maintains a low computation cost and is significantly more efficient than Li et al.'s protocol [35].

Next, we further evaluate the performance of data outsourcing under different deduplication ratios. In the experiment, the number of files is fixed at 100, each with a size of 10 MB. As shown in Fig. 4(a), as the file duplication ratio increases, the number of initial uploads decreases, while the number of subsequent uploads increases. From the comparison of Fig. 4(b) and (c), the computation cost of the initial uploading is inversely proportional to the ratio of file duplication, whereas the computation cost of the subsequent uploading is proportional to it. This behavior is expected, since higher duplication ratios lead to fewer initial-upload requests and more subsequent-upload requests that need to be processed. Fig. 4(d) shows that the data deduplication scheme can substantially reduce the overall system workload, especially as the file duplication ratio increases. This is because the computational overhead of subsequent uploads is significantly lower than that of initial uploads. It can also be observed that, compared with the other two schemes, our BTAD still exhibits favorable performance advantages due to its simpler architecture.

*2) On-Chain Computation Cost:* As depicted in Fig. 5, the on-chain computation cost is evaluated in terms of initial uploading, subsequent uploading and integrity auditing, which

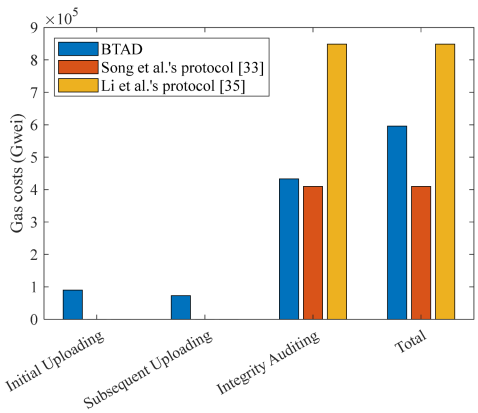


Fig. 5. Comparisons of on-chain computation cost for different schemes.

is measured through gas usage. During the phases of initial uploading and subsequent uploading, both the protocols of [33] and [35] do not need to maintain an on-chain mapping table to which users a file belongs. They thus have no gas cost, whereas our BTAD has. Notably, the reason why our BTAD introduces an on-chain mapping table is to allow users to go offline in front of the DPD and fundamentally resolve the potential disputes on deduplication patterns. During the phase of integrity auditing, our scheme has a low gas cost due to its simpler construction. Besides, the gas cost of the protocol of [35] significantly exceeds the other two protocols because it needs to publish the additional proof of deduplication pattern on the blockchain. Note that although the protocol of [33] has the lowest gas consumption, it comes at the cost of sacrificing the security capability to resist the DPD. In summary, our BTAD not only effectively enhances security but also keeps gas consumption at a low level.

## VIII. Conclusion

This paper first proposes a blockchain-based protocol of transparent integrity auditing and secure deduplication over encrypted cloud storage, denoted as BTAD, which truly realizes the protection of the OP and the resistance to the DPD. Specifically, we propose an integrity auditing mechanism by performing the random masking technique on the file tags and the integrity proofs for preventing the OP of the users from the TPA, and a transparent deduplication mechanism by using the on-chain deduplication pattern jointly maintained by both the CSP and its users for resisting the DPD of the CSP. Furthermore, we also propose a hash-based probabilistic PoW in order to resist the DFA. After that, we strictly prove that our BTAD achieves the property of data privacy, data unforgeability, OP protection, and resistance to DFA and DPD. At last, we evaluate the performance of our BTAD through theoretical analysis and experimental simulation, which further validates its superiority. Under commercial cloud environments, the proposed BTAD can enhance users' trust in cloud storage services with the enhanced security. This increased trust is expected to encourage long-term use of cloud storage, thereby promoting sustainable development of the commercial cloud environments. In the future, interesting directions include extending the proposed protocol to support richer application scenarios, such as dynamic updates, keyword-based auditing, and encrypted search.

## References


[1] N. Sehgal and P. Bhatt, *Cloud Computing Concepts and Practices*. Cham, Switzerland: Springer, 2018.

[2] *Top Threats to Cloud Computing Pandemic Eleven*. Seattle, WA, USA: Cloud Security Alliance, 2022, pp. 1–48.

[3] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2007, pp. 598–609.

[4] C. Wang, S. S. M. Chow, Q. Wang, K. Ren, and W. Lou, "Privacy-preserving public auditing for secure cloud storage," *IEEE Trans. Comput.*, vol. 62, no. 2, pp. 362–375, Feb. 2013.

[5] B. Sengupta, A. Dixit, and S. Ruj, "Secure cloud storage with data dynamics using secure network coding techniques," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 2090–2101, Third Quarter 2022.

[6] C. Hahn, H. Kwon, D. Kim, and J. Hur, "Enabling fast public auditing and data dynamics in cloud services," *IEEE Trans. Serv. Comput.*, vol. 15, no. 4, pp. 2047–2059, Jul./Aug. 2022.

[7] J. Yu, K. Ren, and C. Wang, "Enabling cloud storage auditing with verifiable outsourcing of key updates," *IEEE Trans. Inf. Forensics Secur.*, vol. 11, no. 6, pp. 1362–1375, Jun. 2016.

[8] J. Yu and H. Wang, "Strong key-exposure resilient auditing for secure cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 8, pp. 1931–1940, Aug. 2017.

[9] Y. Xu, J. Ren, Y. Zhang, C. Zhang, B. Shen, and Y. Zhang, "Blockchain empowered arbitrable data auditing scheme for network storage as a service," *IEEE Trans. Serv. Comput.*, vol. 13, no. 2, pp. 289–300, Mar./Apr. 2020.

[10] H. Wang, Q. Wang, and D. He, "Blockchain-based private provable data possession," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2379–2389, Sep./Oct. 2021.

[11] Y. Yang, Y. Chen, and F. Chen, "A compressive integrity auditing protocol for secure cloud storage," *ACM/IEEE Trans. Netw.*, vol. 29, no. 3, pp. 1197–1209, 2021.

[12] Y. Yang, Y. Chen, F. Chen, and J. Chen, "An efficient identity-based provable data possession protocol with compressed cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1359–1371, 2022.

[13] B. Wang, B. Li, and H. Li, "Panda: Public auditing for shared data with efficient user revocation in the cloud," *IEEE Trans. Serv. Comput.*, vol. 8, no. 1, pp. 92–106, Jan./Feb. 2015.

[14] Y. Zhang, J. Yu, R. Hao, C. Wang, and K. Ren, "Enabling efficient user revocation in identity-based cloud storage auditing for shared Big Data," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 3, pp. 608–619, May/Jun. 2020.

[15] H. Wang, D. He, J. Yu, and Z. Wang, "Incentive and unconditionally anonymous ID-based public provable data possession," *IEEE Trans. Service Comput.*, vol. 12, no. 5, pp. 824–835, Sep./Oct. 2019.

[16] L. Huang et al., "IPANM: Incentive public auditing scheme for non-manager groups in clouds," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 936–952, Mar./Apr. 2022.

[17] G. Reinsel and J. Gantz, "The digital universe decade-are you ready?," 2010. [Online]. Available: https://www.ifap.ru/pr/2010/n100507a.pdf

[18] D. T. Meyer and W. J. Bolosky, "A study of practical deduplication," *ACM Trans. Storage*, vol. 7, no. 4, pp. 1–20, 2012.

[19] J. R. Douceur, A. Adya, W. J. Bolosky, P. Simon, and M. Theimer, "Reclaiming space from duplicate files in a serverless distributed file system," in *Proc. Int. Conf. Distrib. Comput. Syst.*, 2002, pp. 617–624.

[20] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Message-locked encryption and secure deduplication," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2013, pp. 296–312.

[21] R. Chen, Y. Mu, G. Yang, and F. Guo, "BL-MLE: Block-level message-locked encryption for secure large file deduplication," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 12, pp. 2643–2652, Dec. 2015.

[22] Y. Zhao and S. S. M. Chow, "Updatable block-level message-locked encryption," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 4, pp. 1620–1631, Jul./Aug. 2021.

[23] B. Mihir, K. Sriram, and R. Thomas, "DupLESS: Server-aided encryption for deduplicated storage," in *Proc. USENIX Secur. Symp.*, 2013, pp. 179–194.

[24] J. Li, X. Chen, F. Xhafa, and L. Barolli, "Secure deduplication storage systems supporting keyword search," *J. Comput. System Sci.*, vol. 81, no. 8, pp. 1532–1541, 2015.

[25] J. Li et al., "Secure distributed deduplication systems with improved reliability," *IEEE Trans. Comput.*, vol. 64, no. 12, pp. 3569–3579, Dec. 2015.

[26] J. Li, J. Wu, L. Chen, and J. Li, "Blockchain-based secure and reliable distributed deduplication scheme," in *Proc. Int. Conf. Algorithms Architectures Parallel Process.*, 2018, pp. 393–405.

[27] J. Yuan and S. Yu, "Secure and constant cost public cloud storage auditing with deduplication," in *Proc. Int. Conf. Commun. Netw. Secur.*, 2013, pp. 145–153.

[28] J. Li, J. Li, D. Xie, and Z. Cai, "Secure auditing and deduplicating data in cloud," *IEEE Trans. Comput.*, vol. 65, no. 8, pp. 2386–2396, Aug. 2016.

[29] X. Liu, W. Sun, W. Lou, Q. Pei, and Y. Zhang, "One-tag checker: Message-locked integrity auditing on encrypted cloud deduplication storage," in *Proc. Int. Conf. Comput. Commun.*, 2017, pp. 1–9.

[30] H. Yuan, X. Chen, J. Wang, J. Yuan, H. Yan, and W. Susilo, "Blockchain-based public auditing and secure deduplication with fair arbitration," *Inf. Sci.*, vol. 541, pp. 409–425, 2020.

[31] G. Tian et al., "Blockchain-based secure deduplication and shared auditing in decentralized storage," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 3941–3954, Nov./Dec. 2022.

[32] Q. Zhang, D. Sui, J. Cui, C. Gu, and H. Zhong, "Efficient integrity auditing mechanism with secure deduplication for blockchain storage," *IEEE Trans. Comput.*, vol. 72, no. 8, pp. 2365–2376, Aug. 2023.

[33] M. Song, Z. Hua, Y. Zheng, H. Huang, and X. Jia, "Blockchain-based deduplication and integrity auditing over encrypted cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 4928–4945, Nov./Dec. 2023.

[34] L. Han, G. Xu, and Q. Xie, "Comments on an efficient identity-based provable data possession protocol with compressed cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 3934–3935, 2023.

[35] S. Li, C. Xu, Y. Zhang, Y. Du, and K. Chen, "Blockchain-based transparent integrity auditing and encrypted deduplication for cloud storage," *IEEE Trans. Serv. Comput.*, vol. 16, no. 1, pp. 134–146, Jan./Feb. 2023.

[36] G. Wood et al., "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[37] S. Halevi, D. Harnik, B. Pinkas, and A. Shulman-Peleg, "Proofs of ownership in remote storage systems," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2011, pp. 491–500.

[38] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://pdos.csail.mit.edu/6.824/papers/bitcoin.pdf

[39] D. Boneh and M. Franklin, "Identity-based encryption from the weil pairing," *SIAM J. Comput.*, vol. 32, no. 3, pp. 586–615, 2003.

[40] N. P. Smart, "The discrete logarithm problem on elliptic curves of trace one," *J. Cryptol.*, vol. 12, no. 3, pp. 193–196, 1999.

[41] D. Angelo and I. Vincenzo, "jPBC: Java pairing based cryptography," 2023. [Online]. Available: http://gas.dia.unisa.it/projects/jpbc/